

81. Regexp: Unicode `\u` flag aur `\p{...}`

JS strings Unicode use karte hain. Zyaadatar characters 2 bytes ke hote hain, par isse sirf 65536 characters hi ban sakte hain. Isliye kuch rare characters (jaise `x`, 😊), kuch hieroglyphs) **4 bytes** se encode hote hain.

Character	Unicode	Bytes
a	<code>0x0061</code>	2
~	<code>0x2248</code>	2
<code>x</code>	<code>0x1d4b3</code>	4
😊	<code>0x1f604</code>	4

JS jab bani thi tab 4-byte characters the hi nahi, isliye kuch features unhe galat handle karte hain:

```
js
'😊'.length;    // 2
'x'.length;    // 2
```

Ye "surrogate pair" hai (4 bytes ko do 2-byte characters ki tarah gina jaata hai).

Regexps bhi by default 4-byte characters ko 2 alag characters maante hain, aur strings ki tarah kai ajeeb results aa sakte hain. Strings ke ulta regexps me `\u` flag hai jo ye theek karta hai:

- 4-byte characters sahi se (ek character ki tarah) handle hote hain
- **Unicode property search** (`\p{...}`) available ho jaati hai

Unicode properties `\p{...}`

Unicode ke har character ki kai properties hoti hain (wo kis "category" ka hai, etc.). Jaise `Letter` property = kisi bhi language ki alphabet ka letter, `Number` = digit (Arabic, Chinese...).

`\p{...}` se property ke hisaab se search kar sakte hain. `\u` flag zaruri hai.

```
js
```

```
let str = "A ზ ა";
```

```
str.match(/\p{L}/gu); // A,ზ,ა (English, Georgian, Korean letters)
str.match(/\p{L}/g); // null (bina "u" ke \p kaam nahi karta)
```

`\p{Letter}` aur `\p{L}` same hain (alias). Almost har property ka chhota alias hota hai.

Main categories aur subcategories

- **Letter** `[L]`: lowercase `[Ll]`, modifier `[Lm]`, titlecase `[Lt]`, uppercase `[Lu]`, other `[Lo]`
- **Number** `[N]`: decimal digit `[Nd]`, letter number `[Nl]`, other `[No]`
- **Punctuation** `[P]`: connector `[Pc]`, dash `[Pd]`, initial quote `[Pi]`, final quote `[Pf]`, open `[Ps]`, close `[Pe]`, other `[Po]`
- **Mark** `[M]` (accents etc): spacing combining `[Mc]`, enclosing `[Me]`, non-spacing `[Mn]`
- **Symbol** `[S]`: currency `[Sc]`, modifier `[Sk]`, math `[Sm]`, other `[So]`
- **Separator** `[Z]`: line `[Zl]`, paragraph `[Zp]`, space `[Zs]`
- **Other** `[C]`: control `[Cc]`, format `[Cf]`, not assigned `[Cn]`, private use `[Co]`, surrogate `[Cs]`

Jaise lowercase letters `\p{Ll}`, punctuation `\p{P}`.

Derived categories bhi hain: `Alphabetic` (`Alpha`) (letters `[L]` + letter numbers `[Nl]` jaise XII + kuch aur), `Hex_Digit` (`[0-9]`, `[a-f]`), etc.

Poori list ke liye references: unicode.org ke `character.jsp`, `list-unicodeset.jsp`, `PropertyValueAliases.txt`.

Example: hex numbers

```
js
let regexp = /x\p{Hex_Digit}\p{Hex_Digit}/u;
"number: xAF".match(regexp); // xAF
```

Example: Chinese hieroglyphs

`Script` property (writing system): `Cyrillic`, `Greek`, `Arabic`, `Han` (Chinese)... `Script=<value>` ya `sc=<value>`:

```
js
let regexp = /\p{sc=Han}/gu;
let str = `Hello Привет 你好 123_456`;
str.match(regexp); // 你,好
```

Example: currency

\$, €, ¥ jaise characters `\p{Currency_Symbol}` (alias `\p{Sc}`):

js

```
let regexp = /\p{Sc}\d/gu;  
'Prices: $2, €1, ¥9'.match(regexp);    // $2,€1,¥9
```

(Kai digits wale numbers ke liye quantifiers, agle chapters me.)

Summary

`u` flag Unicode support on karta hai, yaani:

1. 4-byte characters **ek** character ki tarah handle hote hain.
2. Search me **Unicode properties** `\p{...}` use ho sakti hain (kisi language ke words, quotes, currencies jaise special characters dhundhne ke liye).